

Chapter 17: Transactions

Outline

Transaction Concept

A Simple Transaction Model

Concurrent Executions

Serializability

Recoverability

Transaction Isolation Levels

Transaction Definition in SQL

Transaction Concept

A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.

E.g. transaction to transfer \$50 from account A to account B:

In SQL:

```
update account set balance=balance-50
               where account_number=A;
update account set balance=balance+50
               where account_number=B;
commit;
```

Two main issues to deal with:

Failures of various kinds, such as hardware failures and system crashes

Concurrent execution of multiple transactions

ACID Properties

Atomicity (原子性). Either all operations of the transaction are properly reflected in the database or none are.

Consistency (一致性). Execution of a transaction in isolation preserves the consistency of the database.

Isolation (隔离性). Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.

That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.

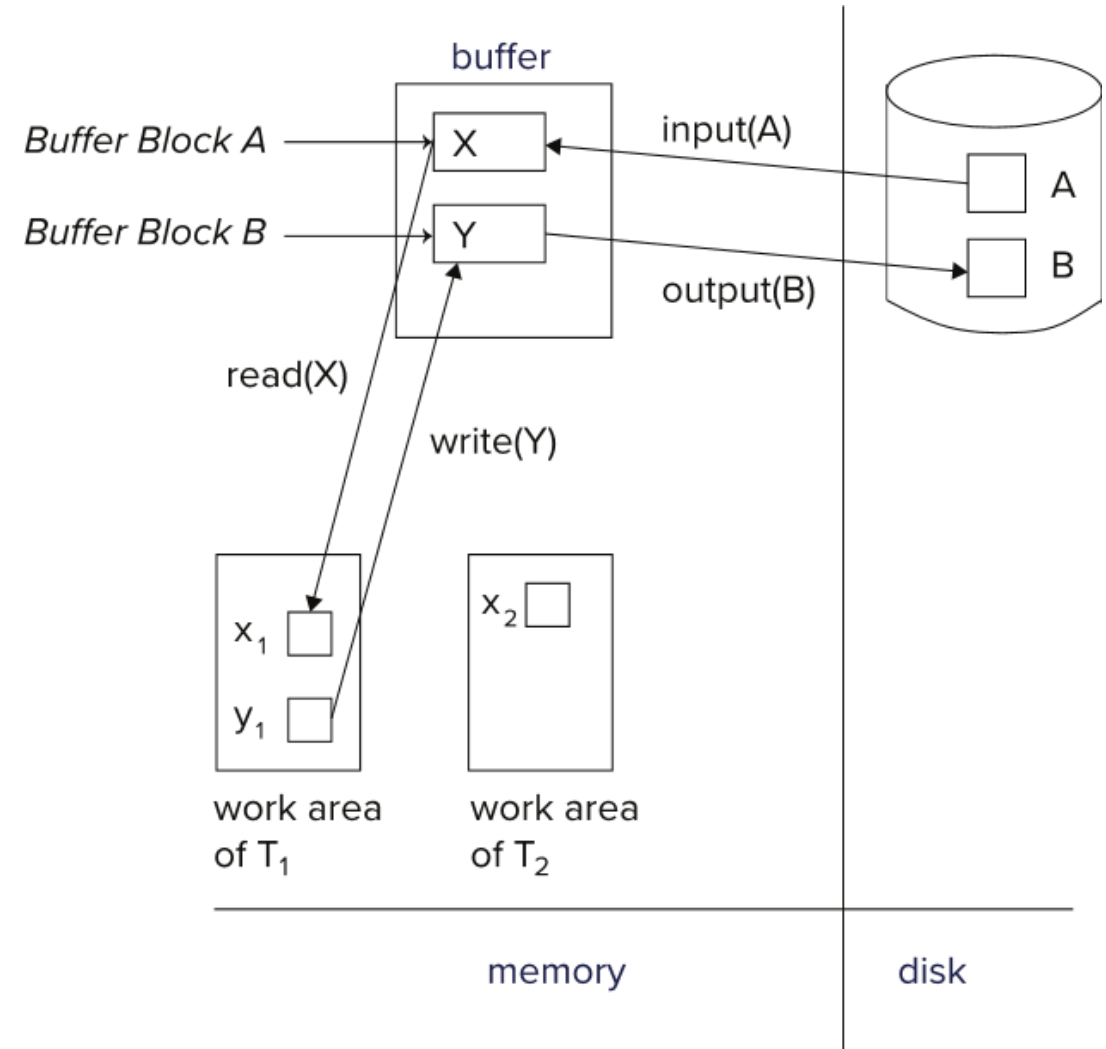
Durability (持久性). After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

A Simple Transaction Model

Transactions access data using two operations:

read(X), which transfers the data item X from the database to a variable, also called X, in a work area in main memory belonging to the transaction that executed the read operation.

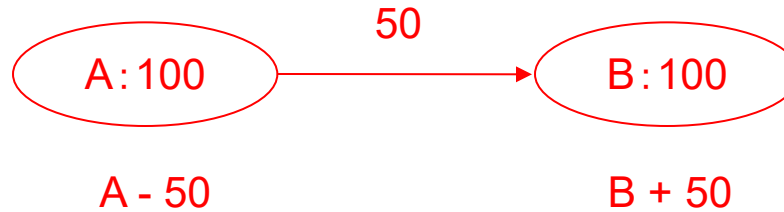
write(X), which transfers the value in the variable X in the main memory work area of the transaction that executed the write operation to the data item X in database.



Example of Fund Transfer

Transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
-
4. **read**(B)
5. $B := B + 50$
6. **write**(B)



Atomicity requirement

if the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state

- ▶ Failure could be due to software or hardware

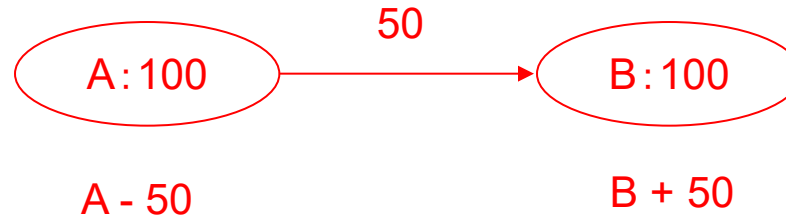
the system should ensure that updates of a partially executed transaction are not reflected in the database

Durability requirement — once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

Example of Fund Transfer (Cont.)

Transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
-
4. **read**(B)
5. $B := B + 50$
6. **write**(B)



Consistency requirement in above example:

the sum of A and B is unchanged by the execution of the transaction

In general, consistency requirements include

- ▶ **Explicitly** (显式) specified integrity constraints :
 - primary keys , foreign keys
- ▶ **Implicit** (隐式) integrity constraints, e.g. :
 - sum of balances of all accounts minus sum of loan amounts must equal value of cash-in-hand

A transaction must see a consistent database.

During transaction execution the database may be temporarily inconsistent.

When the transaction completes successfully the database must be consistent. Erroneous transaction logic can lead to inconsistency

Example of Fund Transfer (Cont.)

Isolation requirement — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1	T2
1. read(A)	
2. $A := A - 50$	
3. write(A)	
-----	read(A), read(B), print(A+B)
4. read(B)	
5. $B := B + 50$	
6. write(B)	

Isolation can be ensured trivially by running transactions **serially**(串行) that is, one after the other.

However, executing multiple transactions concurrently has significant benefits, as we will see later.

Transaction State

A transaction must be in one of the following states:

Active – the initial state; the transaction stays in this state while it is executing

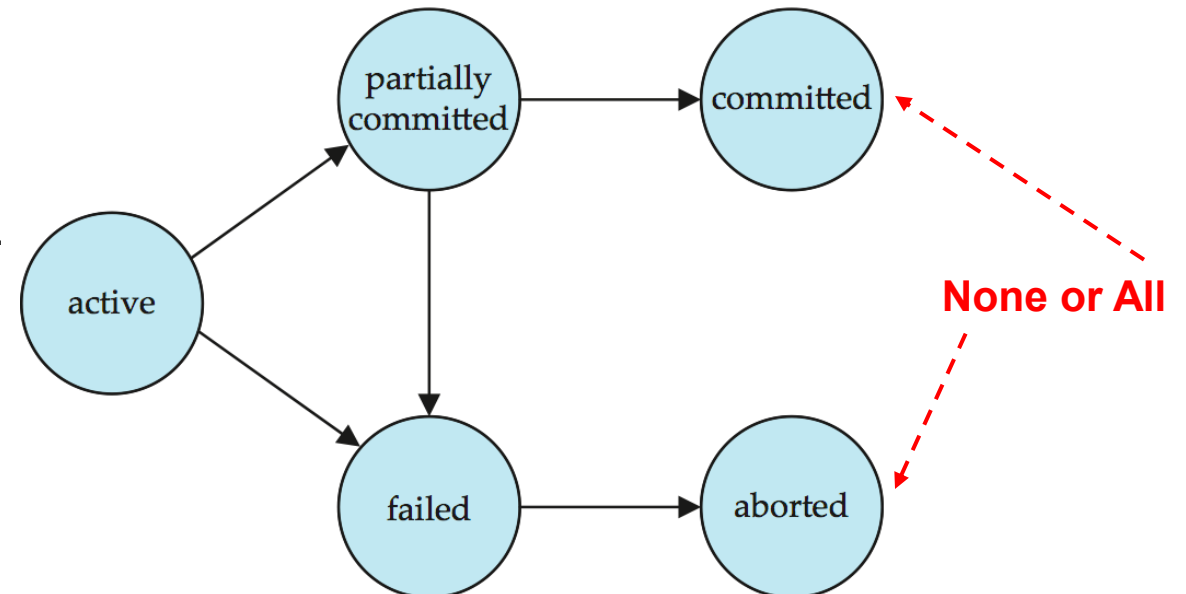
Partially committed – after the final statement has been executed.

Failed -- after the discovery that normal execution can no longer proceed.

Aborted – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:

- ▶ restart the transaction
- ▶ kill the transaction

Committed – after successful completion.



Concurrent Executions

Multiple transactions are allowed to run concurrently in the system.
Advantages are:

increased processor and disk utilization, leading to better transaction *throughput*

- ▶ E.g. one transaction can be using the CPU while another is reading from or writing to the disk

reduced average response time for transactions: short transactions need not wait behind long ones.

Concurrent Executions

Anomalies in Concurrent Executions

Lost Update（丢失修改）

Dirty Read（读脏数据）

Unrepeatable Read（不可重复读）

Phantom Problem（幽灵问题）

Anomalies in Concurrent Executions-

Lost Update(丢失修改)

T1	T2
Read A (100)	
	Read A (100)
A = A-1 (99)	
	A=A-1 (99)
Write A (99)	
	Write A (99)

Anomalies in Concurrent Executions-

Dirty Read (读脏数据)

T1	T2
Read A (100)	
A = A-1 (99)	
Write A (99)	
	Read A (99)
	A=A-1 (98)
rollback	
	Write A (98)
	commit

Anomalies in Concurrent Executions-

Unrepeatable Read

T1	T2
Read A (100)	
	Read A (100)
	A=A-1 (99)
	Write A (99)
Read A (99)	

Anomalies with Concurrent Executions- Phantom Problem

T1	T2
select * from student where age=18 (100 records with age=18)	
	insert into student(id, gender, age) values('008','M', 18) (Add a new record with age=18)
select * from student where age=18 (101 records with age=18 !!!)	

Schedules

Schedule – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed

- a schedule for a set of transactions must consist of all instructions of those transactions

- must preserve the order in which the instructions appear in each individual transaction.

A transaction that successfully completes its execution will have a **commit** instructions as the last statement

- by default transaction assumed to execute commit instruction as its last step

A transaction that fails to successfully complete its execution will have an **abort** instruction as the last statement

Schedule 1

Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .

A **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 2

- A **serial** schedule where T_2 is followed by T_1

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 3

Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is **equivalent** to Schedule 1.

T_1	T_2
read (A) 100	
$A := A - 50$ 50	
write (A) 50	
	read (A) 50
	$temp := A * 0.1$ 5
	$A := A - temp$
	write (A) 45
read (B) 100	
$B := B + 50$ 150	
write (B) 150	
commit	
	read (B) 150
	$B := B + temp$
	write (B) 155
	commit

In Schedules 1, 2 and 3, the sum $A + B$ is preserved.

Schedule 4

The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
read (A) 100	
$A := A - 50$ 50	
	read (A) 100
	$temp := A * 0.1$ 10
	$A := A - temp$
	write (A) ← 90
	read (B) 100
write (A) ← 50	
read (B) 100	
$B := B + 50$	
write (B) 150	
commit	
	$B := B + temp$
	write (B) 110
	commit

Serializability(可串行化)

Basic Assumption – Each transaction preserves database consistency.

Thus serial execution of a set of transactions preserves database consistency.

A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule.

Different forms of schedule equivalence give rise to the notions of:

- **conflict serializability(冲突可串行化)**
- **view serializability(视图可串行化)**

Conflicting Instructions

Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .

1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
2. $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
3. $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict
4. $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict

Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.

If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

Conflict Serializability (冲突可串行化)

If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.

We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule

Conflict Serializability (Cont.)

Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A) write (A)	read (A) write (A)
read (B) write (B)	read (B) write (B)

Schedule 3

T_1	T_2
read (A) write (A) read (B) write (B)	read (A) write (A) read (B) write (B)

Schedule 6

Conflict Serializability (Cont.)

Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.

Testing for Serializability

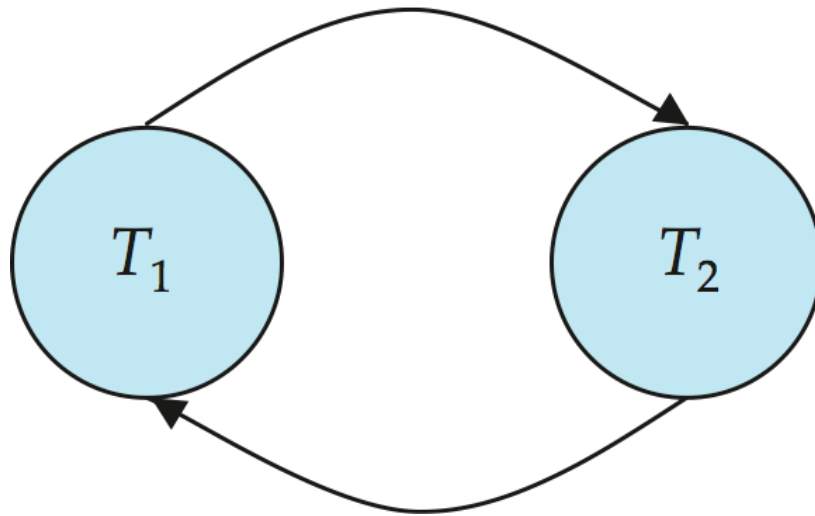
Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$

Precedence graph(前驱图) — a directed graph where the vertices are the transactions (names).

We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.

We may label the arc by the item that was accessed.

Example 1



Test for Conflict Serializability

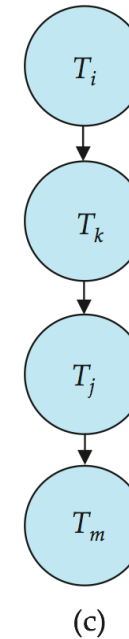
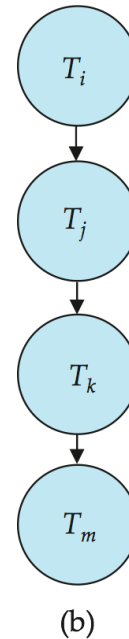
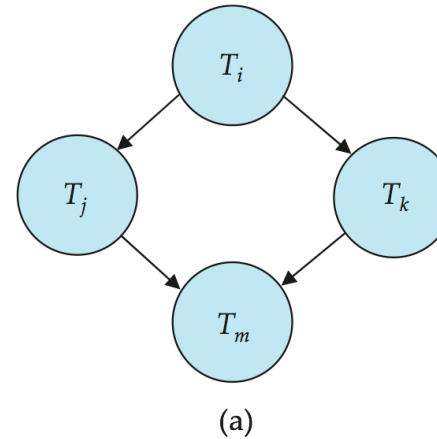
A schedule is conflict serializable if and only if its precedence graph is acyclic.

Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.

(Better algorithms take order $n + e$ where e is the number of edges.)

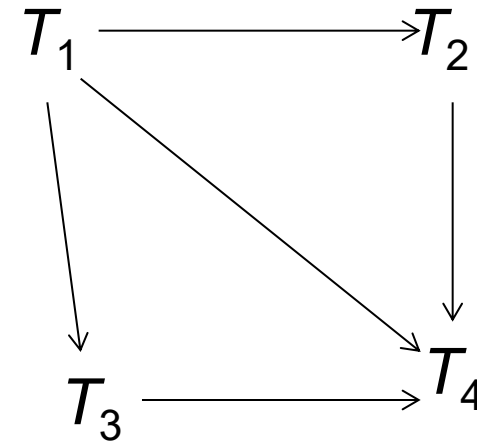
If precedence graph is acyclic, the **serializability order** can be obtained by a *topological sorting* of the graph.

This is a linear order consistent with the partial order of the graph.



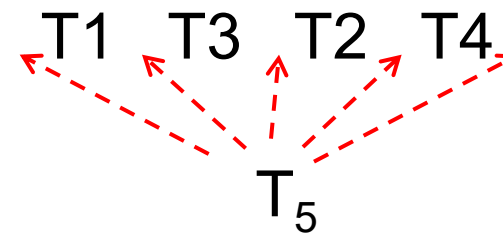
Example Schedule (Schedule A) + Precedence Graph

T_1	T_2	T_3	T_4	T_5
read(Y) read(Z)	read(X)			read(V) read(W) read(W)
	read(Y) write(Y)	write(Z)		
read(U)			read(Y) write(Y) read(Z) write(Z)	
read(U) write(U)				



T_5

T1 T2 T3 T4



Different forms of Serializability

Different forms of schedule equivalence give rise to different forms of serializability.

■ conflict serializability

(冲突可串行化)

- same order of the conflict operation pairs

■ view serializability

(视图可串行化)

- same data content of the corresponding read (Q) operations
- same data content (Q) of the final write (Q) operations

- T1 read the initial value of A. T1 read the initial value of B.
- T2 read the value of A that was produced by T1. T2 read the value of B that was produced by T1.
- T2 performs the final write(A). T2 performs the final write(B).

Schedule 1

T_1	T_2
read (A) ◆ $A := A - 50$ write (A) read (B) ◆ $B := B + 50$ write (B) commit	read (A) ← $temp := A * 0.1$ $A := A - temp$ write (A) ● read (B) ← $B := B + temp$ write (B) ● commit

Schedule 3

T_1	T_2
read (A) ◆ $A := A - 50$ write (A) read (B) ◆ $B := B + 50$ write (B) commit	read (A) ← $temp := A * 0.1$ $A := A - temp$ write (A) ● read (B) ← $B := B + temp$ write (B) ● commit

View Serializability (视图可串行化)

Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q ,

1. If in schedule S , transaction T_i reads the initial value of Q , then in schedule S' also transaction T_i must read the initial value of Q .
2. If in schedule S transaction T_i executes **read(Q)**, and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write(Q)** operation of transaction T_j .
3. The transaction (if any) that performs the **final write(Q)** operation in schedule S must also perform the final **write(Q)** operation in schedule S' .

View Serializability (Cont.)

A schedule S is **view serializable** if it is view equivalent to a serial schedule.

Every conflict serializable schedule is also view serializable.

Below is a schedule which is **view-serializable** but **not conflict serializable**.

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		write (Q)

What serial schedule is above equivalent to?

$T_{27} - T_{28} - T_{29}$

Other Notions of Serializability

The schedule below produces same outcome as the serial schedule $\langle T_1, T_5 \rangle$, yet is not conflict equivalent or view equivalent to it.

T_1	T_5
read (A) $A := A - 50$ write (A)	
	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)

- $(B-10)+50 = (B+50)-10$
- In our example, the final result is the same as that of the serial schedule $\langle T_1, T_5 \rangle$, because of the mathematical fact that **the increment and decrement operations are commutative**.
- The general case is not so easy since a transaction may be expressed as a complex SQL statement, a Java program with JDBC calls, etc.

Determining such equivalence requires analysis of operations other than read and write.

Recoverable Schedules

Recoverable schedule (可恢复调度) — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .

The following schedule (Schedule 11) is not recoverable if T_9 commits immediately after the read

T_8	T_9
read (A)	
write (A) ←	
	read (A) ←
	commit
read (B)	

If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

Cascading Rollbacks

Cascading rollback – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A) ←		
	read (A) ←	
	write (A) ←	
		read (A) ←
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

Can lead to the undoing of a significant amount of work

Cascadeless Schedules

Cascadeless schedules (无级联调度) — cascading rollbacks cannot occur; for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .

Every cascadeless schedule is also recoverable

It is desirable to restrict the schedules to those that are cascadeless

Concurrency Control & Serializability

A database must provide a mechanism that will ensure that all possible schedules are

either **conflict** or **view serializable**, and
are **recoverable** and preferably **cascadeless**

Testing a schedule for serializability *after* it has executed is a little too late!

Goal – to develop **concurrency control protocols** that will assure serializability.

Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view **serializable**, and are **recoverable and cascadeless** .

Concurrency control protocols generally do not examine the precedence graph as it is being created

Instead a protocol imposes a discipline that avoids nonserializable schedules.

Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.

Tests for serializability help us understand why a concurrency control protocol is correct.

Weak Levels of Consistency

Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable

E.g. a **read-only transaction** that wants to get an approximate total balance of all accounts

E.g. **database statistics** computed for query optimization can be approximate

Such transactions need not be serializable with respect to other transactions

Tradeoff accuracy for performance

Transaction Isolation Levels

Serializable — default

Repeatable read — only committed records to be read, repeated reads of same record must return same value. However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.

Read committed — only committed records can be read, but successive reads of record may return different (but committed) values.

Read uncommitted — even uncommitted records may be read.

Lower degrees of consistency useful for gathering approximate information about the database

Warning: some database systems do not ensure serializable schedules by default

E.g. Oracle and PostgreSQL by default support a level of consistency called **snapshot isolation** (not part of the SQL standard)

Transaction Definition in SQL

In SQL, a transaction begins implicitly.

A transaction in SQL ends by:

Commit work commits current transaction and begins a new one.

Rollback work causes current transaction to abort.

In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully

Implicit commit can be turned off by a database directive

- ▶ E.g., in JDBC -- `connection.setAutoCommit(false);`

Isolation level can be set at database level

Isolation level can be changed at start of transaction

- ▶ E.g. In SQL **set transaction isolation level serializable**
- ▶ E.g. in JDBC -- `connection.setTransactionIsolation(`

`Connection.TRANSACTION_SERIALIZABLE)`

Transaction Examples

Transaction Boundaries

Booking one ticket vs multiple tickets?



Transaction Examples



Transaction Boundaries

Booking tickets & paying order - combining vs separating ?

< 未完成 剩余: 29分44秒

为了保护您和他人的身体健康, 请旅客们在车站和列车上全程佩戴口罩, 感谢支持配合!

第一程

龙游 08:29 G46 经停站 徐州东 12:34
历时4小时5分
发车时间: 2021年05月31日 周一

孙建伶 (中国居民身份证) 成人票 ¥ 379
二等座 11车 06D号 [退改说明](#)

第二程

徐州东 14:14 G3171 经停站 银川 21:34
历时7小时20分
发车时间: 2021年05月31日 周一

孙建伶 (中国居民身份证) 成人票 ¥ 633
二等座 06车 14D号 [退改说明](#)

中国铁路保险, 全方位守护您的旅程安全

铁路乘意险 ¥3/人

¥ 1012 取消订单 立即支付

未完成 剩余: 29分14秒

为了保护您和他人的身体健康, 请旅客们在车站和列车上全程佩戴口罩, 感谢支持配合!

第一程

龙游 08:29 G46 经停站 徐州东 12:34
历时4小时5分
发车时间: 2021年05月31日 周一

孙建伶 (中国居民身份证) 成人票 ¥ 379
二等座 11车 06D号 [退改说明](#)

温馨提示

一天内3次申请车票成功后取消订单, 当日将不能在12306继续购票!

关闭 确定取消

第二程

徐州东 14:14 G3171 经停站 银川 21:34
历时7小时20分
发车时间: 2021年05月31日 周一

孙建伶 (中国居民身份证) 成人票 ¥ 633
二等座 06车 14D号 [退改说明](#)

Concurrency Control Protocols

Lock-Based Protocols

Lock on whole database vs lock on items

How long to hold lock?

Shared vs exclusive locks

Timestamp-Based Protocols

Transaction timestamp assigned e.g. when a transaction begins

Data items store two timestamps

- ▶ Read timestamp
- ▶ Write timestamp

Timestamps are used to detect out of order accesses

Validation-Based Protocols

Optimistic concurrency control

Low rate of conflicts among transactions

Each transaction must go through 3 phases:

- ▶ Read phase → Validation phase → Write phase

End of Chapter 17